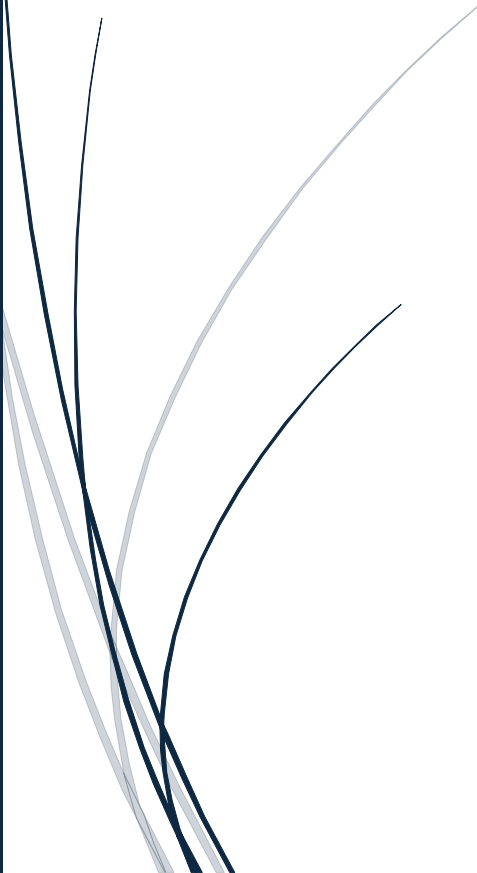




12/3/2025

Classwork Lab 5

Broken Bridge: Multi-Cloud CTF



Levine, Miles L

MLEVIN56 ENPM665 0101

2. Section A: Mission Evidence (Screenshots) Please provide screenshots to verify your progress. If you could not complete a step, provide a screenshot of the error you encountered.

The Recon: Screenshot of the terminal command used to list/find the bucket.

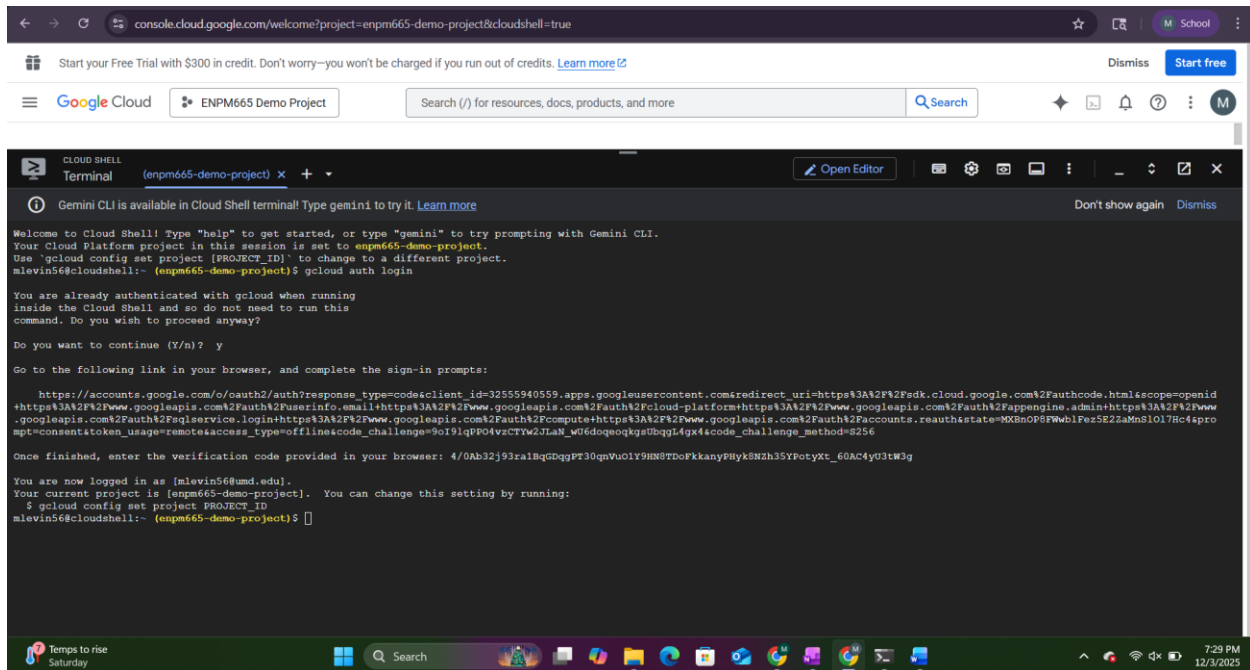
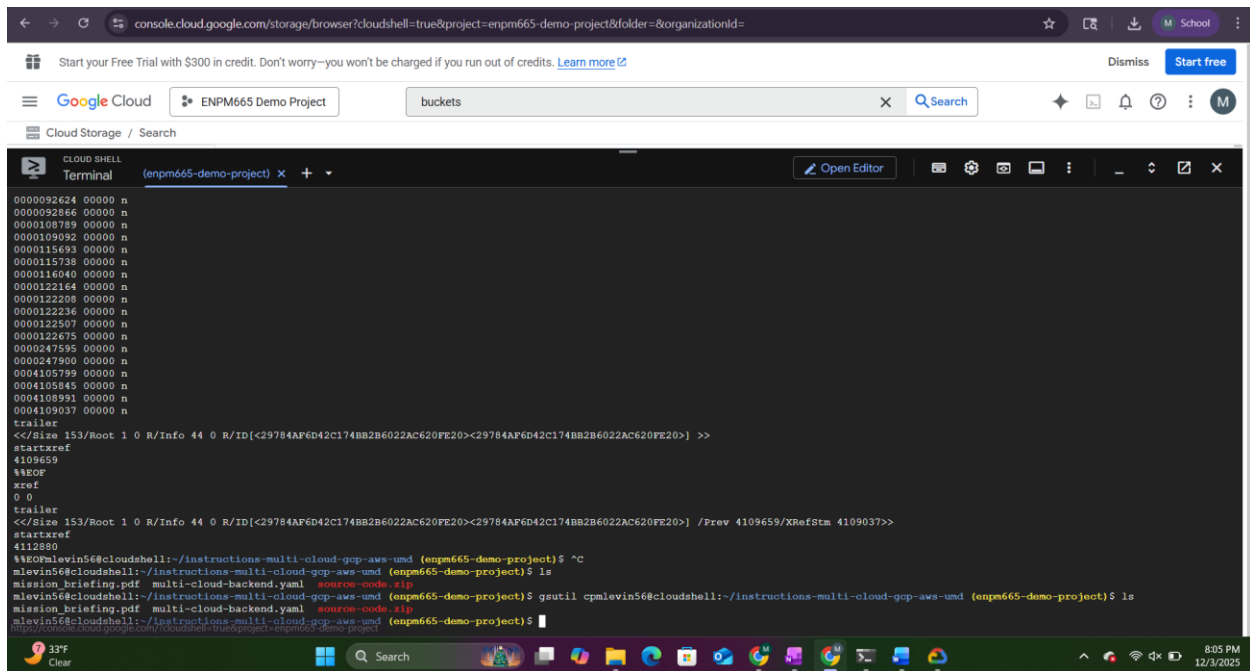


Figure 1 Phase 0: Getting Started (The Onboarding)



```
C:\WINDOWS\SYSTEM32\cmd x + v
Your current project is [enpm665-demo-project]. You can change this setting by running:
$ gcloud config set project PROJECT_ID

C:\Users\miles\AppData\Local\Google\Cloud SDK>gcloud config list
[accessibility]
screen_reader = True
[core]
account = mlewin56@umd.edu
disable_usage_reporting = True
project = enpm665-demo-project

Your active configuration is: [default]

C:\Users\miles\AppData\Local\Google\Cloud SDK>gsutil ls
gs://enpm665-test-bucket-1/
gs://instructions-multi-cloud-gcp-aws-umd/

C:\Users\miles\AppData\Local\Google\Cloud SDK>gsutil ls gs://instructions-multi-cloud-gcp-aws-umd/
gs://instructions-multi-cloud-gcp-aws-umd/mission_briefing.pdf
gs://instructions-multi-cloud-gcp-aws-umd/multi-cloud-backend.yaml
gs://instructions-multi-cloud-gcp-aws-umd/source-code.zip

C:\Users\miles\AppData\Local\Google\Cloud SDK>gsutil cp -r gs://instructions-multi-cloud-gcp-aws-umd/ .
Copying gs://instructions-multi-cloud-gcp-aws-umd/mission_briefing.pdf... - [1 files][ 3.9 MiB/ 3.9 MiB] Copy
/ [0 files][ 0.0 B/ 3.9 MiB]
ing gs://instructions-multi-cloud-gcp-aws-umd/multi-cloud-backend.yaml... \ [2 files][ 3.9 MiB/ 3.9 MiB] Copy
- [1 files][ 3.9 MiB/ 3.9 MiB]
ing gs://instructions-multi-cloud-gcp-aws-umd/source-code.zip... \ [2 files][ 3.9 MiB/ 3.9 MiB]
\ [2 files][ 3.9 MiB/ 3.9 MiB] \ [3 files][ 3.9 MiB/ 3.9 MiB]
Operation completed over 3 objects/3.9 MiB.

C:\Users\miles\AppData\Local\Google\Cloud SDK>ls
'ls' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\miles\AppData\Local\Google\Cloud SDK>dir
Volume in drive C is Blade 15 SSD
Volume Serial Number is 62F0-FF3F

Directory of C:\Users\miles\AppData\Local\Google\Cloud SDK
```

```
ssh.cloud.google.com/v2/ssh/projects/enpm665-demo-project/zones/us-central1-c/instances/instruction-vm-mi... — □ ×
ssh.cloud.google.com/v2/ssh/projects/enpm665-demo-project/zones/us-central1-c/instances/instruction-vm-mil...

SSH-in-browser UPLOAD FILE DOWNLOAD FILE ! ⌨ ⚙

sed. at /usr/share/perl5/Debconf/FrontEnd/Dialog.pm line 78.)
debconf: falling back to frontend: Readline
Scanning processes...
Scanning candidates...
Scanning linux images...

Running kernel seems to be up-to-date.

Restarting services...
Daemons using outdated libraries
-----

1. networkd-dispatcher.service 2. unattended-upgrades.service

(Enter the items or ranges you want to select, separated by spaces.)

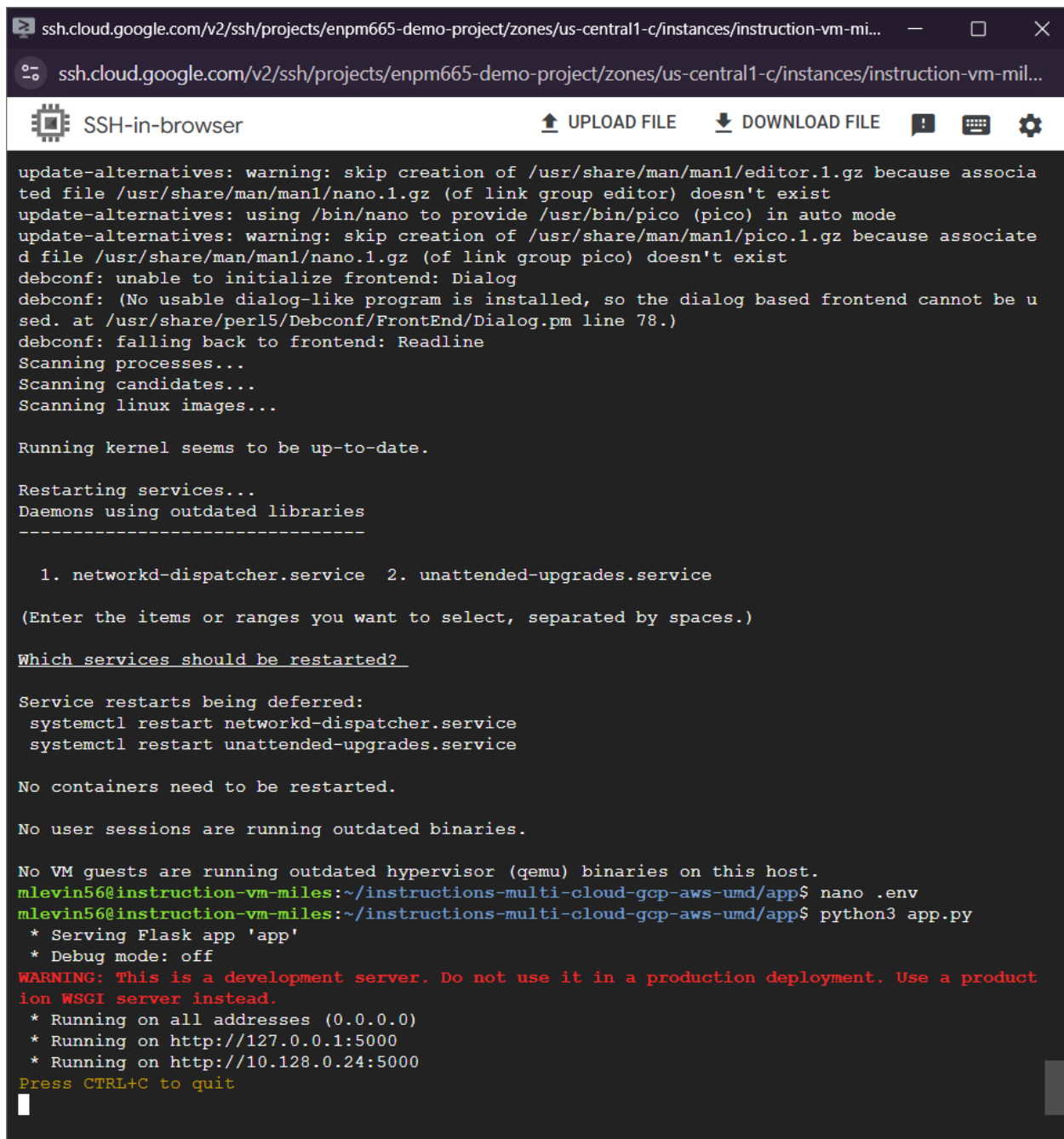
Which services should be restarted?

Service restarts being deferred:
systemctl restart networkd-dispatcher.service
systemctl restart unattended-upgrades.service

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
mlevin56@instruction-vm-miles:~$ ls
instructions-multi-cloud-gcp-aws-umd snap
mlevin56@instruction-vm-miles:~$ cd instructions-multi-cloud-gcp-aws-umd
mlevin56@instruction-vm-miles:~/instructions-multi-cloud-gcp-aws-umd$ unzip source-code.zip -d
app
Archive: source-code.zip
  creating: app/static/
  inflating: app/static/style.css
  creating: app/templates/
  inflating: app/templates/index.html
  inflating: app/templates/mission.html
  inflating: app/.env.example
  inflating: app/app.py
  inflating: app/requirements.txt
mlevin56@instruction-vm-miles:~/instructions-multi-cloud-gcp-aws-umd$ cd app
mlevin56@instruction-vm-miles:~/instructions-multi-cloud-gcp-aws-umd/app$ ls
app.py requirements.txt static templates
mlevin56@instruction-vm-miles:~/instructions-multi-cloud-gcp-aws-umd/app$
```

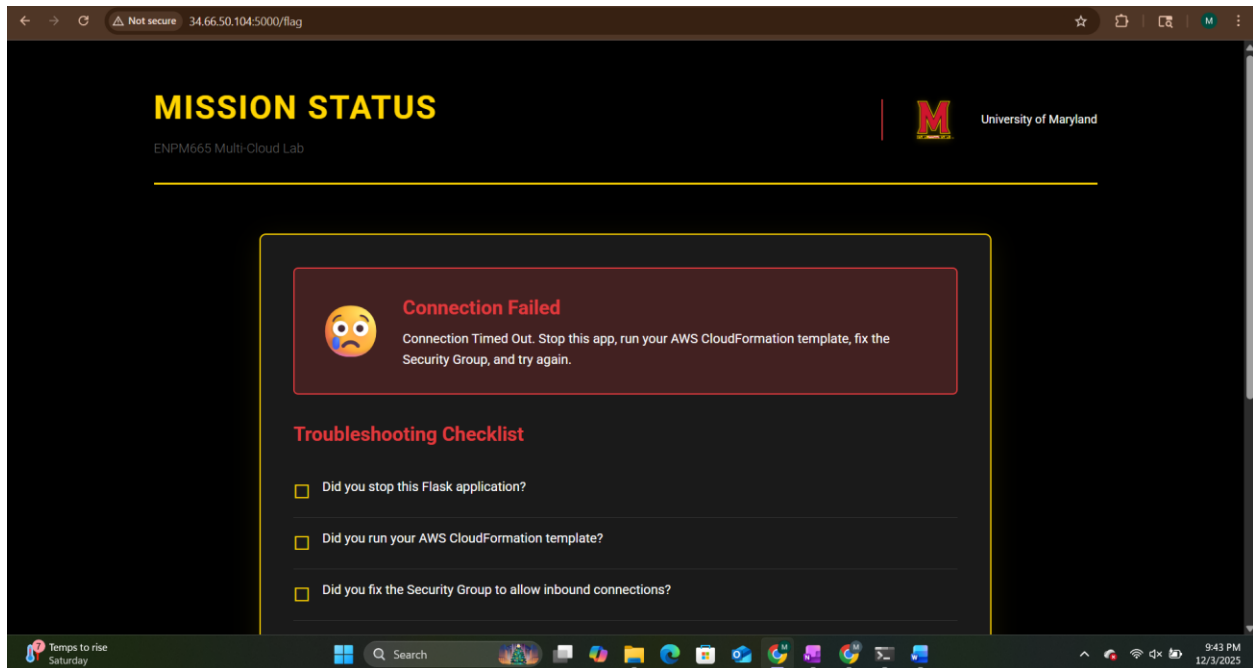


The screenshot shows a web browser window with the address bar displaying `ssh.cloud.google.com/v2/ssh/projects/enpm665-demo-project/zones/us-central1-c/instances/instruction-vm-mil...`. The browser interface includes a title bar, a toolbar with icons for uploading and downloading files, and a terminal window titled "SSH-in-browser".

The terminal output shows the following sequence of events:

- System update warnings for `update-alternatives` regarding `/usr/share/man/man1/nano.1.gz` and `/usr/share/man/man1/pico.1.gz`.
- Debconf messages indicating that the dialog-based frontend cannot be used and falling back to the Readline frontend.
- Scanning processes, candidates, and Linux images.
- Running kernel seems to be up-to-date.
- Restarting services...
- Daemons using outdated libraries.
- A list of services to be restarted: `1. networkd-dispatcher.service 2. unattended-upgrades.service`.
- Service restarts being deferred.
- Systemctl commands to restart `networkd-dispatcher.service` and `unattended-upgrades.service`.
- No containers need to be restarted.
- No user sessions are running outdated binaries.
- No VM guests are running outdated hypervisor (qemu) binaries on this host.
- Execution of `nano .env` and `python3 app.py`.
- Flask application output: `* Serving Flask app 'app'`, `* Debug mode: off`, and a warning: `WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.`
- Flask application running on `0.0.0.0`, `http://127.0.0.1:5000`, and `http://10.128.0.24:5000`.
- Press `CTRL+C` to quit.

The Failure: Screenshot of the "Sad Face" / Connection Timeout on the Web App.



The Fix: Screenshot of the AWS Security Group Inbound Rule you created.

The Success: Screenshot of the "Mission Success" page with the Flag (or your best attempt).

Operational Analysis

What Went Well

I authenticated into the GCP project and reached the correct environment. I created the VM, installed the required tools, and downloaded the mission files from the storage bucket. I deployed the frontend application and confirmed that it loaded in my browser after creating the firewall rule. This showed that the VM was reachable and the frontend was functioning.

I deployed the AWS backend using CloudFormation and confirmed that the RDS instance was created with the expected settings. I reviewed the endpoint, port, and security group information. These steps matched what was shown in the lecture recording, and they helped me understand the structure of the multi cloud environment.

What Did Not Go Well

I first attempted the lab on my own. When the frontend attempted to connect to the RDS instance, it returned a connection failure screen. I rewatched Josh's lecture recording to confirm the correct process for setting up the environment file, reviewing the endpoint, and adjusting the

security group. I updated the AWS Security Group to allow my GCP VM external IP. I checked the RDS settings and verified that public access was enabled.

I reached the point where I learned that my environment file was missing the `DB_NAME` variable. I had created my own `.env` file even though the source code included a `.env.example` file with the correct variables. Leaving out `DB_NAME` could have caused the connection failure because the application uses that value to connect to the database. I stopped troubleshooting at that stage and did not test the fix. I deleted my VM to recreate a clean instance and later lost the permissions needed to create a new VM, which blocked further progress.

What I Would Do Differently

I would use the environment file that was included in the source code instead of creating a new one. I would confirm all required variables before launching the application. I would follow the lecture steps more closely when setting up the environment file. I would avoid deleting the VM until I confirmed that my permissions were stable. I would save important values and notes before restarting the application or changing files.

Security Posture Assessment

IAM

The environment relied on broad permissions. I would apply least privilege for each role and restrict deployment access. I would replace static credentials with managed identities or temporary tokens.

Secrets Management

The environment used plain text environment files for sensitive values. I would move these values into a secrets manager with rotation and access control. This reduces exposure and prevents credential leaks.

Network Architecture

The setup exposed both the VM and the RDS instance to the public internet. I would replace this design with private connectivity between clouds. VPC peering or a VPN tunnel would keep database traffic inside private networks and reduce the need for wide open inbound rules. I would use targeted firewall rules and limit inbound access as much as possible.

Summary

I reached the application in my browser and confirmed that the frontend worked. The issue occurred during the database connection process. The lab reinforced the importance of environment variables, security group configuration, and understanding how cloud components communicate. Even though I did not finish the final phase, the experience improved my understanding of multi cloud deployment and debugging.